Function parameters and return types

Department of Computer Science
University of Pretoria

4 April 2024

What do we already know about functions?

-
-
-

What do we already know about functions?

All functions have a:

- name
- formal parameter list
- return type
- function body

- We have already considered all combinations of formal parameters and return types in L13.
  - with no formal parameters and no return type (`void`)
  - with one or more formal parameters and no return type
  - with no formal parameters and a return type
  - with one or more formal parameters and a return type

- No parameters and no return type –
  `void functionA();`
- One/more parameters and no return type –
  `void functionB(int);`
- No parameters and a return type –
  `bool functionB();`
- One/more formal parameters and a return type –
  `bool functionB(int, float);`

Consider the following example:

```
#include <iostream>
using namespace std;

void assignValue(int a, int b) {
    a = b;
}

int main() {
    int x = 10;
    int y = 20;
    assignValue(x,y);
    return 0;
}
```

Questions: What are the values of:

- `x` and `y` before and after the call to `assignValue` in the main?
- `a` and `b` before the assignment and after the assignment in `assignValue`?

If we change the example as follows:

```
#include <iostream>
using namespace std;

void assignValue(int a, int b = 30) {
    a = b;
}

int main() {
    int x = 10;
    int y = 20;
    assignValue(x,y);
    return 0;
}
```

Questions: What are the values of:

- `x` and `y` before and after the call to `assignValue` in the main?
- `a` and `b` before the assignment and after the assignment in `assignValue`?

What about the following?

```
#include <iostream>
using namespace std;

void assignValue(int a, int b = 30) {
    a = b;
}

int main() {
    int x = 10;
    int y;
    assignValue(x,y);
    return 0;
}
```

Questions: What are the values of:

- `x` and `y` before and after the call to `assignValue` in the main?
- `a` and `b` before the assignment and after the assignment in `assignValue`?

Can we guarantee that `y` is always the same value?

Will the following compile?

```
#include <iostream>
using namespace std;

void assignValue(int a, int b = 30) {
    a = b;
}

int main() {
    int x = 10;
    int y;
    assignValue(x);
    return 0;
}
```

What will happen here?

```
#include <iostream>
using namespace std;

void assignValue(int a) {
    a = 40;
}

void assignValue(int a, int b = 30) {
    a = b;
}

int main() {
    int x = 10;
    int y;
    assignValue(x);
    return 0;
}
```

The `assignValue` functions are not overloaded in this example.

```
AssignValue_5.cpp:15:5: error: call to 'assignValue' is ambiguous
    assignValue(x);
    ^~~~~~~~~~~
AssignValue_5.cpp:4:6: note: candidate function
void assignValue(int a) {
     ^
AssignValue_5.cpp:8:6: note: candidate function
void assignValue(int a, int b = 30) {
     ^
1 error generated.
```

An error occurs, Why? How would you fix it to make use of overloaded functions?

Which variant of the function will be called?

- `void`, the absence of type. You cannot create a variable of `void` – Example `TypeVoid.cpp`
- any other valid type – Example `isEqual.cpp`

The TypeVoid.cpp example.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      void aVariable;
7      return 0;
8  }
```

produces the compiler error:

```
TypeVoid.cpp:6:10: error: variable has incomplete type 'void'
    void aVariable;
         ^
1 error generated.
```

What is the output of the `isEqual.cpp` program?

```
1  #include <iostream>
2
3  using namespace std;
4
5  bool isEqual(int a, int b) {
6      return a == b;
7  }
8
9  int main() {
10     int x, y;
11     cin >> x >> y;
12     cout << isEqual(x,y) << endl;
13     return 0;
14 }
```

What would the output be if line 6 was changed from `a == b` to `a = b`?

What is the output?

```
#include <iostream>
using namespace std;

int i;  // Global variable

void someFunction(int k) { // formal parameter to the function
    int j = 1; // Local variable to the function
    cout << i << endl;
    cout << j << endl;
    cout << k << endl;
}

int main() {
    i = 0;
    cout << i << endl
    someFunction();
    return 0;
}
```

- Consider the code to print the truth table entry for the Boolean operators &&, || and !
- The code to print the entry for ! has been given, complete the code for the && and || truth table entries.